Day 30: Timeouts, Deadlines, and Cancellation Propagation

Bounding latency and stopping wasted work in distributed systems.

1. Why Timeouts Matter

- Slow calls hold threads, sockets, and queues
- Timeouts bound tail latency
- Protects service capacity

2. Timeout vs Deadline

TIMEOUT = max wait for one operation

DEADLINE = total remaining end-to-end budget

3. Per-Hop Latency Budgeting

Client Request	Service A	Service B	Service C
Budget 300ms	120ms	100ms	80ms

Each hop checks remaining budget before starting work

4. Cancellation Propagation

- Upstream timeout triggers cancellation
- Downstream work stops early
- Reduces wasted compute and lock contention

5. Fail-Fast Behavior

- If budget is exhausted, fail fast
- Avoids cascading saturation
- Return fast error instead of waiting

6. Day-30 Sample: Deadline Guarded Request Chain

1. Request starts with shared deadline
2. Each service step checks remaining time
3. Slow step consumes budget
4. Timeout occurs → downstream steps cancelled
5. Final result = SUCCESS or TIMEOUT

SUCCESS

TIMEOUT

7. Trade-Offs

- Tight timeouts improve responsiveness but may increase false failures
- Loose timeouts reduce false failures but increase resource pressure
- Tune to SLOs and dependency behavior

8. Interview Takeaways

- Deadlines should propagate end-to-end
- Cancellation protects correctness and capacity
- Per-hop timing and timeout metrics matter

Good timeout design protects latency, capacity, and user experience.

System Design - Day 30

Timeouts, Deadlines, and Cancellation Propagation

Prepared from Day-30 notes and sample implementation

Overview from README.md

Day 30: Timeouts, Deadlines, and Cancellation Propagation

Day 30 focuses on how distributed systems bound latency and stop wasted work when downstream calls are too slow.

It covers:
- timeout and deadline fundamentals
- per-hop latency budgeting
- cancellation propagation across service chains
- fail-fast behavior under budget exhaustion
- partial-work waste reduction
- reliability and UX trade-offs

In short, Day 30 is about enforcing time limits consistently so slow dependencies do not consume resources indefinitely.

Core ideas

Why timeouts matter
- Slow dependencies can hold threads, sockets, and queues for too long.
- Without limits, one slow path can degrade unrelated traffic.
- Timeouts bound tail latency and protect service capacity.

Timeout vs deadline
- `Timeout`: max wait for one operation.
- `Deadline`: absolute remaining budget across the full request.
- Deadline-aware services avoid spending more than the end-to-end latency budget.

Budgeting across hops
- Request budget should be split across downstream calls.
- Each hop checks remaining budget before starting work.
- If budget is exhausted, the call should fail fast.

Cancellation propagation
- When a request times out upstream, downstream work should be cancelled.
- Propagated cancellation reduces wasted compute and lock contention.
- Cancellation signals should be explicit in inter-service contracts.

Trade-offs
- Tight timeouts improve responsiveness but may increase false failures.
- Loose timeouts reduce false failures but increase resource pressure.
- Timeout tuning must align with SLO targets and dependency behavior.

Day-30 sample: Deadline Guarded Request Chain

This repository includes a small timeout/deadline sample in both Python and Java.

Functional requirements
- Execute a request through multiple service steps
- Enforce one shared deadline budget

- Fail fast when remaining budget is insufficient
- Mark downstream steps as cancelled after timeout
- Report timeline and final status (`SUCCESS` or `TIMEOUT`)

High-level components
- `RequestContext`: deadline, elapsed time, cancellation state
- `ServiceStep`: simulated step with fixed processing duration
- `DeadlineExecutor`: orchestrates chain execution with budget checks
- `ExecutionResult`: status and event timeline

Data flow
1. Request starts with total deadline budget
2. Executor checks remaining time before each step
3. Step consumes time from budget
4. On budget exhaustion, executor marks timeout and propagates cancellation
5. Result returns status and per-step timeline for debugging

Project structure

```
Day-30/
README.md
timeout-control-sample/
python/
timeout_control.py
demo.py
java/
TimeoutControlSystem.java
Main.java
```

Run the sample

Python

```
cd timeout-control-sample\python
python demo.py
```

Java

```
cd timeout-control-sample\java
javac Main.java TimeoutControlSystem.java
java Main
```

Interview takeaways
- Deadlines should be propagated end-to-end, not managed independently per hop
- Cancellation is part of correctness and capacity protection
- Timeout policy affects both latency and error rate
- Fail-fast behavior limits cascading saturation under slow dependencies
- Observability needs timeout/cancel metrics and per-hop timing

Next improvements
- Add adaptive timeout tuning from historical latency percentiles
- Add retries with remaining-budget awareness
- Add hedged requests for high-tail dependencies
- Add separate connect/read timeout modeling
- Add tracing IDs and per-hop latency histograms

Reference Implementation

Python and Java sample source files

File: timeout-control-sample/python/demo.py

```python
from timeout_control import DeadlineExecutor, ServiceStep


def print_result(title: str, steps: list[ServiceStep], deadline_ms: int) -> None:
    executor = DeadlineExecutor()
    result = executor.execute(steps=steps, deadline_ms=deadline_ms)
    print(title)
    print(f"  status={result.status.value} elapsed={result.elapsed_ms}ms")
    for line in result.timeline:
        print(" ", line)


def main() -> None:
    fast_chain = [
        ServiceStep("auth-service", 20),
        ServiceStep("profile-service", 25),
        ServiceStep("inventory-service", 30),
    ]
    print_result("successful request chain", fast_chain, deadline_ms=100)

    slow_chain = [
        ServiceStep("auth-service", 20),
        ServiceStep("recommendation-service", 70),
        ServiceStep("pricing-service", 20),
    ]
    print()
    print_result("timeout with cancellation propagation", slow_chain, deadline_ms=80)


if __name__ == "__main__":
    main()
```

File: timeout-control-sample/python/timeout_control.py

```python
from __future__ import annotations

from dataclasses import dataclass
from enum import Enum


class ResultStatus(Enum):
    SUCCESS = "SUCCESS"
    TIMEOUT = "TIMEOUT"


@dataclass(frozen=True)
class ServiceStep:
    name: str
    duration_ms: int


@dataclass
class RequestContext:
    deadline_ms: int
    elapsed_ms: int = 0
    cancelled: bool = False

    def remaining_ms(self) -> int:
        return max(0, self.deadline_ms - self.elapsed_ms)


@dataclass(frozen=True)
class ExecutionResult:
    status: ResultStatus
    elapsed_ms: int
    timeline: list[str]


class DeadlineExecutor:
    def execute(self, steps: list[ServiceStep], deadline_ms: int) -> ExecutionResult:
        if deadline_ms <= 0:
            raise ValueError("deadline_ms must be positive")
        if not steps:
            raise ValueError("steps must be non-empty")

        context = RequestContext(deadline_ms=deadline_ms)
        timeline: list[str] = [f"request started with deadline={deadline_ms}ms"]

        for step in steps:
            if context.cancelled:
                timeline.append(f"{step.name}: skipped (cancelled)")
                continue

            remaining = context.remaining_ms()
            timeline.append(f"{step.name}: remaining_before={remaining}ms")

            if remaining <= 0:
                context.cancelled = True
                timeline.append(f"{step.name}: timeout before execution")
                continue

            if step.duration_ms > remaining:
                context.elapsed_ms += remaining
                context.cancelled = True
                timeline.append(
                    f"{step.name}: timed out after consuming remaining {remaining}ms "
                    f"(needed {step.duration_ms}ms)"
                )
                continue

            context.elapsed_ms += step.duration_ms
            timeline.append(
                f"{step.name}: success duration={step.duration_ms}ms elapsed={context.elapsed_ms}ms"
            )

        status = ResultStatus.TIMEOUT if context.cancelled else ResultStatus.SUCCESS
```

```
    timeline.append(f"request finished status={status.value} elapsed={context.elapsed_ms}ms")
    return ExecutionResult(status=status, elapsed_ms=context.elapsed_ms, timeline=timeline)
```

File: timeout-control-sample/java/Main.java

```java
import java.util.List;

public class Main {
    private static void printResult(String title, List<TimeoutControlSystem.ServiceStep> steps, int deadlineMs)
        TimeoutControlSystem.DeadlineExecutor executor = new TimeoutControlSystem.DeadlineExecutor();
        TimeoutControlSystem.ExecutionResult result = executor.execute(steps, deadlineMs);

        System.out.println(title);
        System.out.println("  status=" + result.getStatus() + " elapsed=" + result.getElapsedMs() + "ms");
        for (String line : result.getTimeline()) {
            System.out.println("  " + line);
        }
    }

    public static void main(String[] args) {
        List<TimeoutControlSystem.ServiceStep> fastChain = List.of(
            new TimeoutControlSystem.ServiceStep("auth-service", 20),
            new TimeoutControlSystem.ServiceStep("profile-service", 25),
            new TimeoutControlSystem.ServiceStep("inventory-service", 30)
        );
        printResult("successful request chain", fastChain, 100);

        List<TimeoutControlSystem.ServiceStep> slowChain = List.of(
            new TimeoutControlSystem.ServiceStep("auth-service", 20),
            new TimeoutControlSystem.ServiceStep("recommendation-service", 70),
            new TimeoutControlSystem.ServiceStep("pricing-service", 20)
        );
        System.out.println();
        printResult("timeout with cancellation propagation", slowChain, 80);
    }
}
```

File: timeout-control-sample/java/TimeoutControlSystem.java

```java
import java.util.ArrayList;
import java.util.List;

public class TimeoutControlSystem {
    public enum ResultStatus {
        SUCCESS,
        TIMEOUT
    }

    public static final class ServiceStep {
        private final String name;
        private final int durationMs;

        public ServiceStep(String name, int durationMs) {
            this.name = name;
            this.durationMs = durationMs;
        }

        public String getName() {
            return name;
        }

        public int getDurationMs() {
            return durationMs;
        }
    }

    public static final class RequestContext {
        private final int deadlineMs;
        private int elapsedMs = 0;
        private boolean cancelled = false;

        public RequestContext(int deadlineMs) {
            this.deadlineMs = deadlineMs;
        }

        public int remainingMs() {
            return Math.max(0, deadlineMs - elapsedMs);
        }

        public int getElapsedMs() {
            return elapsedMs;
        }

        public void addElapsedMs(int delta) {
            elapsedMs += delta;
        }

        public boolean isCancelled() {
            return cancelled;
        }

        public void cancel() {
            cancelled = true;
        }
    }

    public static final class ExecutionResult {
        private final ResultStatus status;
        private final int elapsedMs;
        private final List<String> timeline;

        public ExecutionResult(ResultStatus status, int elapsedMs, List<String> timeline) {
            this.status = status;
            this.elapsedMs = elapsedMs;
            this.timeline = timeline;
        }

        public ResultStatus getStatus() {
            return status;
        }
        public int getElapsedMs() {
```

```java
            return elapsedMs;
        }

        public List<String> getTimeline() {
            return timeline;
        }
    }

    public static final class DeadlineExecutor {
        public ExecutionResult execute(List<ServiceStep> steps, int deadlineMs) {
            if (deadlineMs <= 0) {
                throw new IllegalArgumentException("deadlineMs must be positive");
            }
            if (steps == null || steps.isEmpty()) {
                throw new IllegalArgumentException("steps must be non-empty");
            }

            RequestContext context = new RequestContext(deadlineMs);
            List<String> timeline = new ArrayList<>();
            timeline.add("request started with deadline=" + deadlineMs + "ms");

            for (ServiceStep step : steps) {
                if (context.isCancelled()) {
                    timeline.add(step.getName() + ": skipped (cancelled)");
                    continue;
                }

                int remaining = context.remainingMs();
                timeline.add(step.getName() + ": remaining_before=" + remaining + "ms");

                if (remaining <= 0) {
                    context.cancel();
                    timeline.add(step.getName() + ": timeout before execution");
                    continue;
                }

                if (step.getDurationMs() > remaining) {
                    context.addElapsedMs(remaining);
                    context.cancel();
                    timeline.add(
                        step.getName()
                            + ": timed out after consuming remaining "
                            + remaining
                            + "ms (needed "
                            + step.getDurationMs()
                            + "ms)"
                    );
                    continue;
                }

                context.addElapsedMs(step.getDurationMs());
                timeline.add(
                    step.getName()
                        + ": success duration="
                        + step.getDurationMs()
                        + "ms elapsed="
                        + context.getElapsedMs()
                        + "ms"
                );
            }

            ResultStatus status = context.isCancelled() ? ResultStatus.TIMEOUT : ResultStatus.SUCCESS;
            timeline.add("request finished status=" + status + " elapsed=" + context.getElapsedMs() + "ms");
            return new ExecutionResult(status, context.getElapsedMs(), timeline);
        }
    }
}
```